

Design and Development of a Detachable Tool-Changing Mechanism for a Unitree G1-Inspired Humanoid Robotic Arm

Group 1

Le Hai Trung – V202301000

Ngo Hai Khanh – V202200672

Doan Chi Dung – V202200851

ELEC4060 Robotics and Automation / COMP5050 Robotics

Instructor: Thai Mai Thanh, PhD

Final submission date: 20/06/2026

Contents

1	Introduction	6
1.1	Background	6
1.2	Problem Statement	6
1.3	Project Objectives	6
2	Project Requirements and Design Evolution	7
2.1	Initial Concept	7
2.2	Initial Design Specifications	7
2.3	Design Changes During Development	7
3	Overall System Architecture	8
3.1	System Overview	8
3.2	Mechanical Architecture	9
3.3	Electrical Architecture	9
3.4	Control Architecture	11
3.5	Operating Sequence	11
4	Mechanical Design and Fabrication	11
4.1	Four-DOF Robotic Arm	12
4.2	Joint and Link Design	12
4.3	Wrist Coupling Mechanism	13
4.4	Interchangeable End-Effectors	14
4.4.1	Gripper	15
4.4.2	Drill or Saw Tool	15
4.4.3	Humanoid Hand Module	15
4.5	Rotating Tool Station	16
4.6	Manufacturing and Assembly	17
5	Kinematic Modeling and Motion Planning	17
5.1	Coordinate Frames and Joint Variables	18
5.2	Forward Kinematics	18
5.3	Inverse Kinematics	20
5.4	Joint-to-Servo Mapping	23
5.5	Tool-Change Path Planning	23
6	Electrical and Embedded System	24
6.1	Electronic Components	24
6.2	Power Distribution and Hardware Connections	24
6.3	Electromagnet and Pogo-Pin Interface	25
6.4	Safety and Tuning Considerations	25
7	Control Software and Web Interface	25
7.1	Firmware Structure	26
7.1.1	Configuration & Constants (<code>config.h</code>)	26
7.1.2	Hardware Abstraction & Control (<code>robot_control.cpp / .h</code>)	26
7.1.3	Mathematics & Kinematics Engine (<code>kinematics.cpp / .h</code>)	26
7.1.4	Behavioral Logic (<code>motions.cpp / .h</code>)	26
7.1.5	Main Loop & Serial CLI (<code>main.cpp</code>)	27
7.2	Web Interface	27
7.2.1	Sidebar (Global Settings & Connections)	27

7.2.2	Main Content Tabs	28
7.3	Preset Motions	29
7.4	Automatic Tool-Change State Machine	29
8	Experimental Methodology	30
8.1	Test Setup	30
8.2	Arm Motion Test	30
8.3	Tool Station Indexing Test	31
8.4	Attachment and Release Test	31
8.5	Electrical Contact Test	31
9	Challenges, Limitations, and Improvements	32
9.1	Mechanical Challenges	32
9.2	Electrical Challenges	32
9.3	Control Challenges	33
9.4	Solutions Implemented	33
9.5	Current Limitations	34
10	Project Management and Team Contributions	35
10.1	Development Timeline	35
10.2	Team Contributions	35
11	Conclusion and Future Work	35
11.1	Future Work	35
	Acknowledgment	36
	List of Figures	
1	Initial proposal	8
2	Initial system architecture block diagram.	9
3	Robotic arm CAD modle	10
4	Tool exchanging stations	10
5	CAD model and labeled joints of the final product	12
6	Joint design between the "shoulder" and "arm" (Pitch and Roll)	13
7	Joint design between the "shoulder" and "arm" (Yaw)	13
8	Joint design of the elbow	13
9	Overall view of the wrist coupling, electromagnet, mechanical guide, and pogo-pin interface	14
10	Gripper tool head	15
11	Drill tool head	15
12	Mimic Humanoid hand Modle	16
13	Final Rotating Tool station with all location labled	17
14	Coordinate frames and joint variables used in the kinematic model.	18
15	Power and Hardware wiring diagram.	24
16	Overall block diagram of the code.	25
17	Web UI main page	27
18	Web UI Tool Control	27
19	Web UI stepper control	28
20	Web UI motion control	28

List of Tables

1	Initial project design targets.	7
2	Major changes from the proposal to the final prototype.	8
3	Measured geometric offsets used in the forward kinematic model.	19
4	Main electrical components of the final prototype.	24
5	Challenges and implemented solutions.	34
6	Final project development timeline.	35
7	Final team contributions.	35

1. Introduction

1.1. Background

Humanoid robots benefit from interchangeable tools because a single fixed hand or end-effector cannot efficiently perform every type of task. For example, a gripper may be effective for grasping and moving objects, but it is not suitable for drilling, cutting, suction-based handling, or other specialized operations. Using interchangeable end-effectors allows the robot to select the most appropriate tool for each task without modifying or redesigning the entire robotic arm.

This concept is similar to an automatic tool changer used in CNC machines. A CNC machine can switch between different cutting tools depending on the required machining operation, while the main machine structure remains unchanged. In the same way, a humanoid robot can exchange its end-effector to perform different tasks, improving flexibility, reducing downtime, and expanding the robot's overall range of applications.

1.2. Problem Statement

The main engineering problems addressed in this project are:

- A single end-effector cannot efficiently perform all required tasks.
- Manual replacement increases downtime and reduces robot autonomy.
- The wrist interface must provide repeatable mechanical alignment.
- The tool must remain securely attached during operation.
- Electrical power must be transferred reliably to the attached tool so that tools don't require external power source.
- The robot arm must reach the docking station without collision.
- The exchange sequence need to have high precision to align surface component such as the coupling mechanism and power transmission
- The exchange sequence must be repeatable and controllable.

1.3. Project Objectives

The final project objectives are:

1. Design and fabricate a Unitree G1-inspired four-DOF robotic arm.
2. Develop a detachable wrist interface for interchangeable tools.
3. Design a rotating station that stores and presents different tools.
4. Implement electrical connection through pogo pins and tool fixation using an electromagnet.
5. Develop forward and inverse kinematics for Cartesian positioning.
6. Implement manual, preset, and automatic tool-changing control.
7. Develop a web-based control interface for the complete system.
8. Validate the system through experimental demonstrations.

2. Project Requirements and Design Evolution

2.1. Initial Concept

The original concept consisted of a wrist-side detachable interface, a fixed tool docking station with 3 place corresponding for 3 tools, and multiple task-specific end-effectors. The proposed tool set initially included a pneumatic suction tool, a mini drill, a grapple tool, and an optional humanoid hand module.

The robot was expected to move toward a known docking position, release the currently attached tool, select another tool, establish the mechanical and electrical connection, and activate the appropriate control mode.

2.2. Initial Design Specifications

The initial project targets are summarized in Table 1.

Table 1: Initial project design targets.

Requirement	Target or description
Arm configuration	Unitree G1-inspired robotic arm
Tool-change method	Automatic exchange at a fixed docking station
Mechanical coupling	Repeatable alignment and secure wrist attachment by mechanical fixation like screw based or click-in mechanism
Electrical interface	Shared power and control connection for all tools
Tool control	Simple ON/OFF control for interchangeable modules
Controller	ESP32 or equivalent microcontroller
Manufacturing method	Primarily 3D-printed mechanical components
Control modes	Joint control, Cartesian control, and preset motions

2.3. Design Changes During Development

Several design changes were introduced during the development process. These changes were necessary because of component availability, mechanical complexity, fabrication limitations, and experimental observations.



Figure 1: Initial proposal

Table 2: Major changes from the proposal to the final prototype.

Item	Initial proposal	Final implementation
Robotic arm	Simplified Unitree G1 robotic arm	Four-DOF servo-driven humanoid robotic arm
Wrist locking	Servo-driven or solenoid-driven latch	Electromagnet-based attachment
Electrical connection	Passive electrical contacts	Pogo-pin power interface
Tool station	Fixed multi-slot docking rack	Stepper-driven rotating tool station
Tool set	Suction, drill, grapple, and hand	Drill, grapple, and hand model
Control interface	Shared ON/OFF control panel	Web interface with joint, Cartesian, tool, and station control
Tool selection	Robot moves between fixed docking slots	Rotating station presents the selected tool to the robot
Tool verification	Automatic recognition proposed	The robot remember the tool state with a preset tool at start
Servo control	PWM Integrated board	Breadboard with jumper powered by Keysight DC Generator

3. Overall System Architecture

3.1. System Overview

The complete system consists of the following main modules:

- Four-DOF robotic arm.
- Wrist coupling mechanism.
- Interchangeable end-effectors.

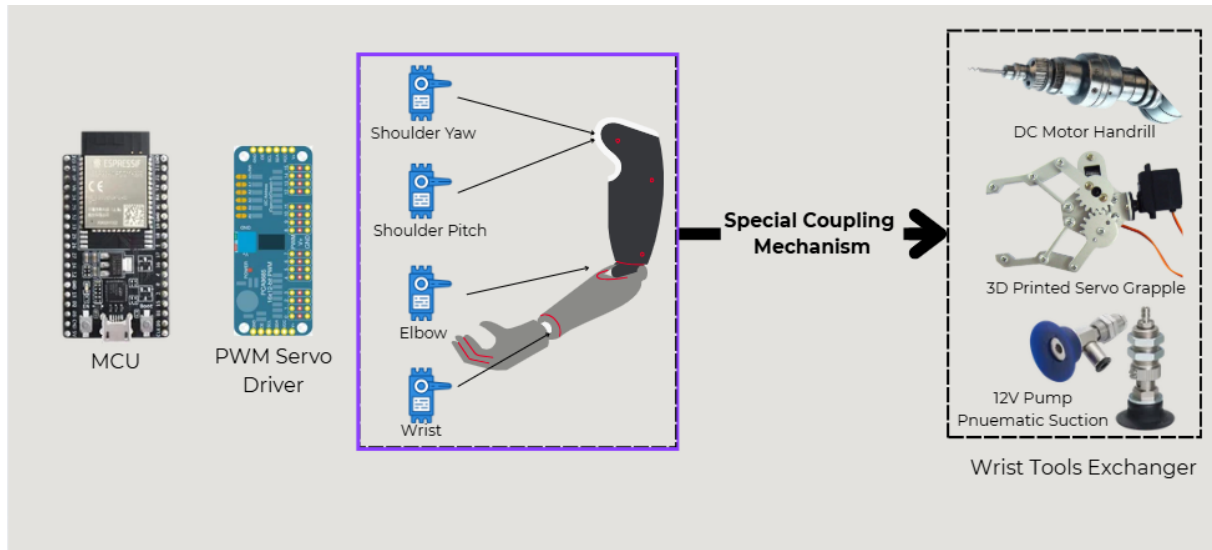


Figure 2: Initial system architecture block diagram.

- Rotating tool station.
- ESP32 controller.
- Servo motors and stepper motor.
- Relay modules.
- Electromagnet.
- Pogo-pin electrical interface.
- Web-based user interface.

3.2. Mechanical Architecture

The mechanical system consists of the robotic arm, the wrist coupler, the end-effector modules, and the rotating tool station. The robotic arm moves the wrist toward the station, while the station presents the selected tool at a known angular position.

Mechanical guidance features are included in the wrist and tool modules to reduce alignment error during attachment. The electromagnet provides the main holding force after the tool reaches the attachment position.

3.3. Electrical Architecture

The electrical architecture contains several voltage rails for different components.

- 12 V supply for the stepper motor and selected tools.
- 6 V supply for the robotic arm servo motors.
- 5 V supply for the ESP32 peripherals, relay modules, electromagnet switching circuit, and pogo-pin tool power.

The power supply grounds must be connected correctly to provide a common reference for the control signals.

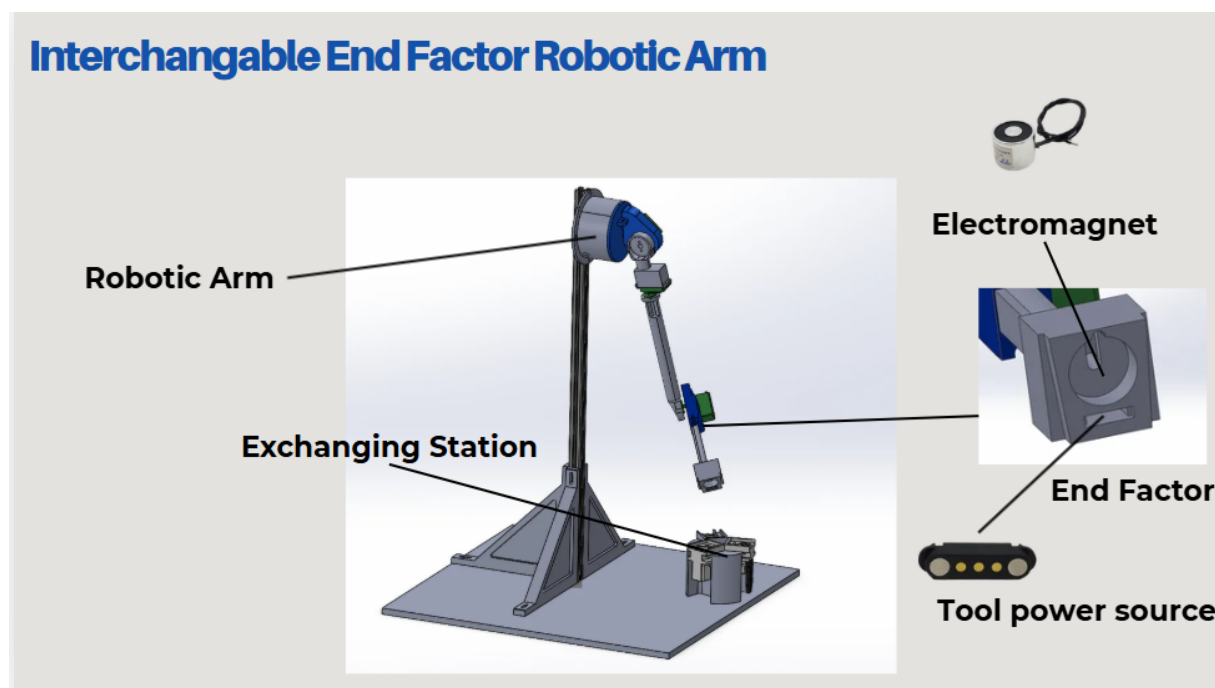


Figure 3: Robotic arm CAD model

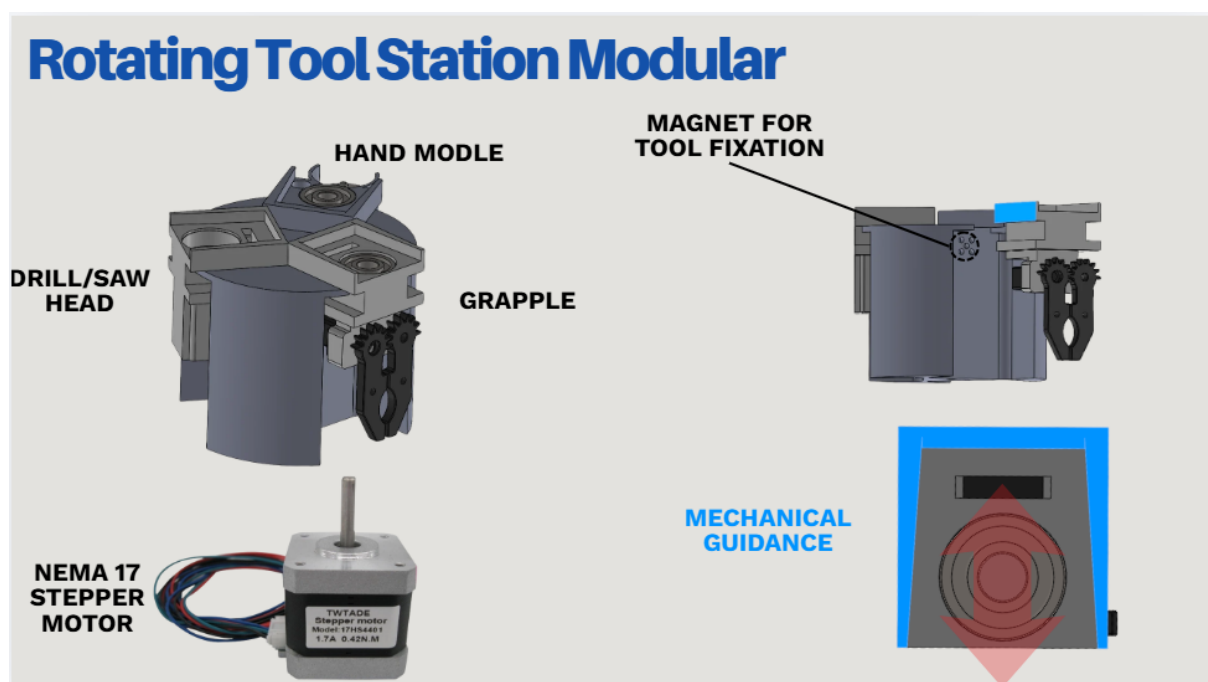


Figure 4: Tool exchanging stations

3.4. Control Architecture

The user sends commands through the web interface. These commands are received by the ESP32, which controls:

- The four arm servo motors, including their angle and speed.
- The stepper motor of the rotating station, including angle and speed.
- The electromagnet relay.
- The pogo-pin power relay.
- The active end-effector (servo signal).

The controller also executes preset arm motions and the automatic tool-changing sequence. Included preset motions are: Hello, Waving, Hand shake, Raise arm, Stroking, Bending, T-Pose, Basic Pick and Place.

3.5. Operating Sequence

The complete operating sequence is:

1. The user selects the desired target tool.
2. The robot moves to the docking approach pose.
3. The controller disables the active tool.
4. The current end-effector is positioned in its station holder.
5. The electromagnet is deactivated to release the current tool.
6. The robot moves away from the station.
7. The station rotates the selected tool to the exchange position.
8. The robot approaches and aligns with the target tool.
9. The electromagnet is activated.
10. The pogo pins establish the tool power connection.
11. The robot retracts from the station.
12. The selected tool is enabled for operation.

4. Mechanical Design and Fabrication

This section presents the mechanical design decisions, fabrication methods, and structural analysis underlying the robotic arm system. The arm was designed to be modular, lightweight, and capable of supporting three distinct interchangeable end-effectors while maintaining sufficient rigidity for precision tasks.

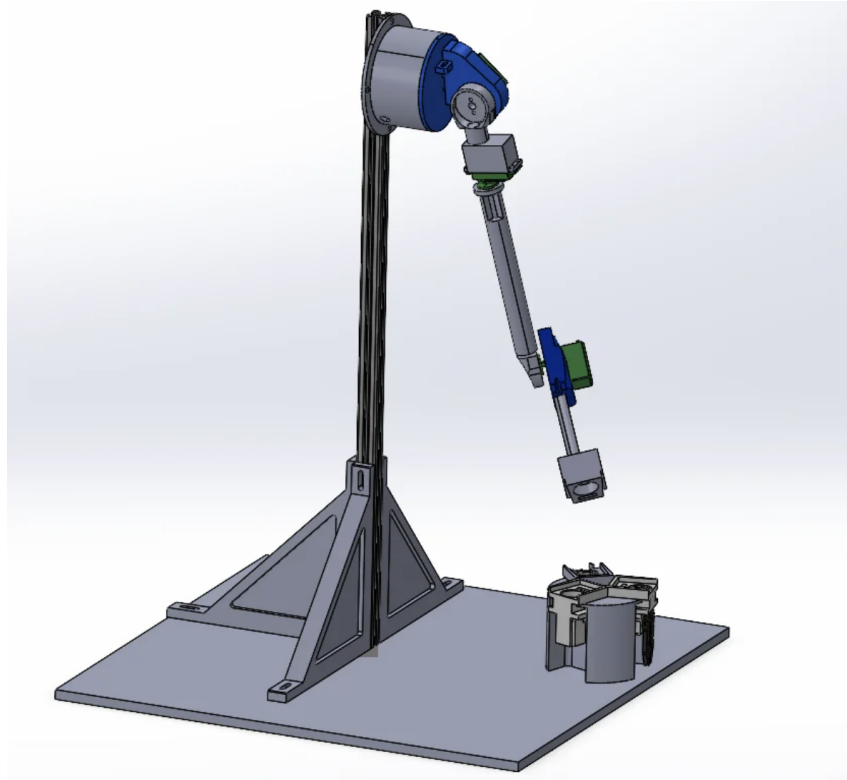


Figure 5: CAD model and labeled joints of the final product

4.1. Four-DOF Robotic Arm

The robotic arm is designed with four degrees of freedom (DOF), providing adequate workspace coverage for the intended applications. The four joints are arranged in a serial kinematic chain, consisting of a base rotation joint, a shoulder pitch joint, an elbow pitch joint, and a wrist rotation joint. This configuration enables the arm to reach a wide three-dimensional workspace while keeping mechanical complexity manageable. The arm was 3D printed using polylactic acid (PLA) plastic and each joint is actuated by a servo motor selected for its torque rating, positional accuracy, and compatibility with the embedded control system.

4.2. Joint and Link Design

The Joint and Link Design of this model is referenced based on a humanoid hand, the joints are then connected to a servo motor and they are optimized for cable management to prevent errors or hardware failures during operation.

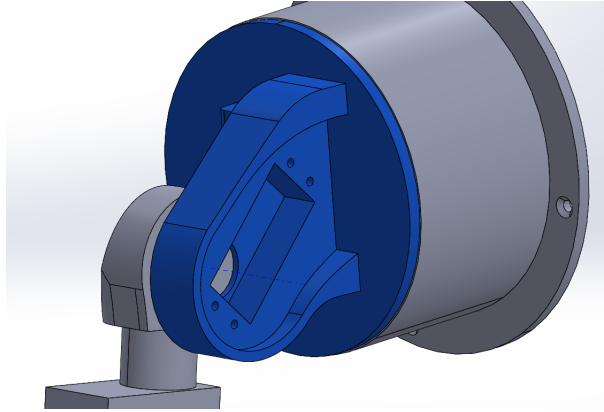


Figure 6: Joint design between the "shoulder" and "arm" (Pitch and Roll)

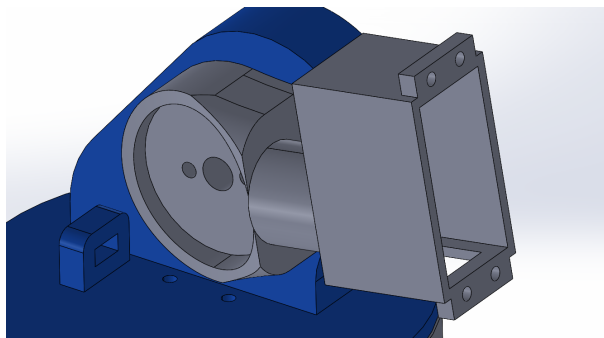


Figure 7: Joint design between the "shoulder" and "arm" (Yaw)

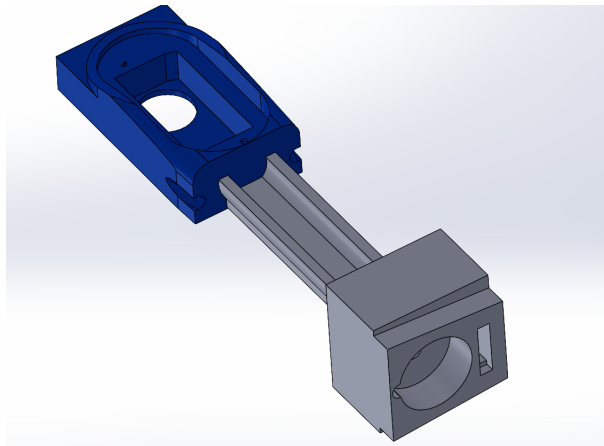


Figure 8: Joint design of the elbow

The links were manufactured primarily through 3D printing. The design was optimized to reduce material use while maintaining sufficient rigidity for the required motions.

4.3. Wrist Coupling Mechanism

4.3 Wrist Coupling Mechanism The wrist coupling mechanism is the central innovation of this project. It provides the mechanical and electrical interface between the robotic arm and any of the interchangeable end-effectors. The mechanism was designed to satisfy the following requirements:

- Secure mechanical attachment capable of withstanding the torques and forces imposed during operation of each tool type.

- Rapid tool change capability, allowing an end-effector swap within approximately 10 seconds without tools.
- Repeatable alignment, ensuring the end-effector re-attaches in the same orientation every time.
- Fully automatic attachment and release under software control, without manual intervention.

The coupling operates as follows. The rotating tool station presents the selected tool at a known angular position. The arm moves to the alignment pose, where mechanical guide features on the wrist housing engage corresponding features on the tool adapter to reduce lateral and rotational misalignment. Once the wrist surface is in contact with the tool steel plate, the electromagnet is energised and pulls the tool firmly against the wrist mounting surface. Simultaneously, the pogo-pin array makes electrical contact with the matching pad on the tool adapter, establishing the power and signal connection. The main components of the wrist coupling are:

- Electromagnet for tool fixation, mounted flush with the wrist face.
- Steel mating plate on each tool adapter, providing the ferromagnetic attachment surface.
- Mechanical alignment guide (funnel or pin-and-slot feature) to correct residual positioning error before magnet activation.
- Anti-rotation feature to prevent the tool from spinning during operation.
- Pogo-pin electrical contacts for power and signal transfer.
- 3D-printed wrist housing enclosing the electromagnet and pogo pins.



Figure 9: Overall view of the wrist coupling, electromagnet, mechanical guide, and pogo-pin interface

4.4. Interchangeable End-Effectors

Three end-effectors were designed and fabricated for this project, each targeting a distinct category of manipulation task. All three share the same mounting interface and are electrically compatible with the wrist coupling mechanism described in **Section 4.3**.

4.4.1 Gripper

The gripper end-effector is a two-finger parallel jaw design intended for grasping and manipulating objects of varying sizes. The fingers are actuated by a single servo motor driving a gear mechanism that opens and closes both fingers symmetrically. The gripper is suitable for objects such as cylindrical parts, small boxes, and electronic components.

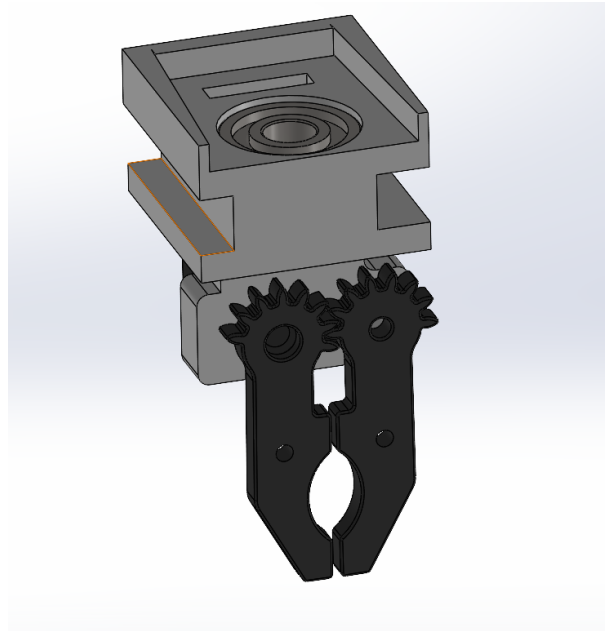


Figure 10: Gripper tool head

4.4.2 Drill or Saw Tool

The drill/saw end-effector integrates a compact DC motor driving a rotary tool attachment. This end-effector is designed for light-duty material processing tasks such as drilling pilot holes or cutting thin sheet material. The motor is powered directly from the 5V line supplied through the pogo-pin interface.

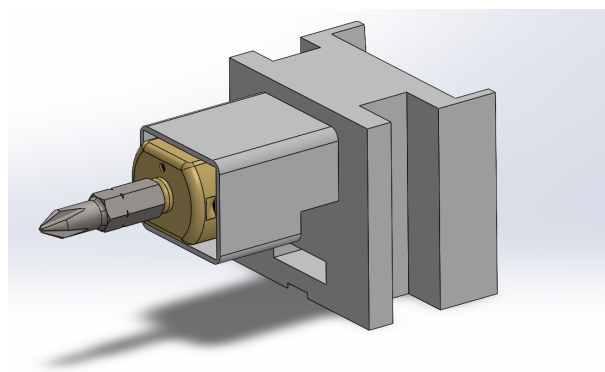


Figure 11: Drill tool head

4.4.3 Humanoid Hand Module

The humanoid hand module is a passive end-effector consisting of a fully 3D-printed hand structure. The hand replicates the general form of a human hand, with five fingers of anatomically proportioned phalanges printed in PLA. The module contains no motors or electronic actuation;

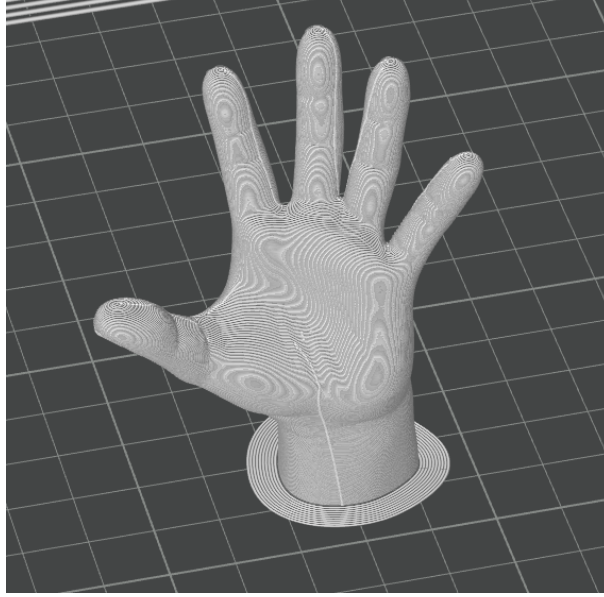


Figure 12: Mimic Humanoid hand Modle

the fingers are fixed in a predetermined pose defined at the design stage, making it suitable for tasks that require a static hand shape such as object support, display, or gesture demonstration. The wrist mount adapter is integrated directly into the base of the palm, incorporating the steel mating plate and pogo-pin pad required for attachment to the wrist coupling. The lightweight all-PLA construction adds minimal load to the wrist joint and electromagnet. This end-effector validates the compatibility of the tool-change system with non-actuated modules and demonstrates the flexibility of the coupling interface.

4.5. Rotating Tool Station

The rotating tool station stores the interchangeable end-effectors and presents the selected tool at a known, repeatable angular position for the arm to approach. The station eliminates the need for the arm to navigate to different fixed docking locations, reducing the complexity of the path-planning problem to a single well-defined exchange pose. The station consists of:

- NEMA 17 stepper motor.
- A4988 stepper motor driver.
- Rotating carousel.
- Individual tool holders.
- Tool-retaining magnets or passive supports.
- Mechanical wrist alignment guide.
- 3D-printed base and housing.

The station angular displacement is controlled by the number of stepper motor pulses.

For a stepper motor with step angle θ_s and microstepping ratio M , the number of pulses required for an angular displacement θ_d is

$$N_{\text{pulse}} = \frac{\theta_d}{\theta_s/M}. \quad (1)$$

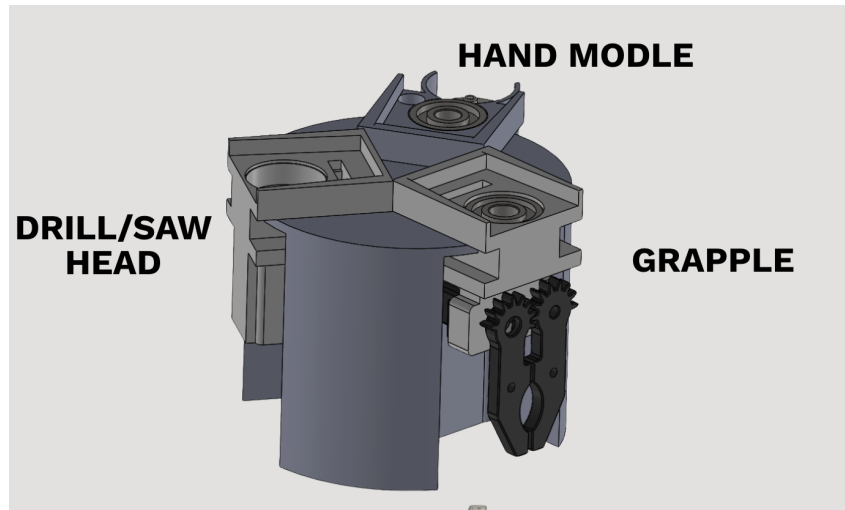


Figure 13: Final Rotating Tool station with all location labled

4.6. Manufacturing and Assembly

All structural components of the arm, wrist coupling, end-effectors, and tool station were designed in SolidWorks and manufactured using fused deposition modelling (FDM) 3D printing with PLA filament.

Several design iterations were required during fabrication. The decision of which tool be included in the final product is needed due to the electromagnetic capacity of the magnet at the end-effector connector. Additionally, several revision on the station that contains the tools is made: instead of making a individual stations for the tools, the team decided on a rotate station to save space and save complexity for the robotic arm to reach each tool station.

5. Kinematic Modeling and Motion Planning

This section presents the mathematical framework used to model the motion of the four-DOF robotic arm. It covers coordinate frame assignment and the forward and inverse kinematics implementations used in the ESP32 firmware, joint-to-servo calibration, workspace analysis, and the interpolated path planning strategy used during tool-change sequences

5.1. Coordinate Frames and Joint Variables

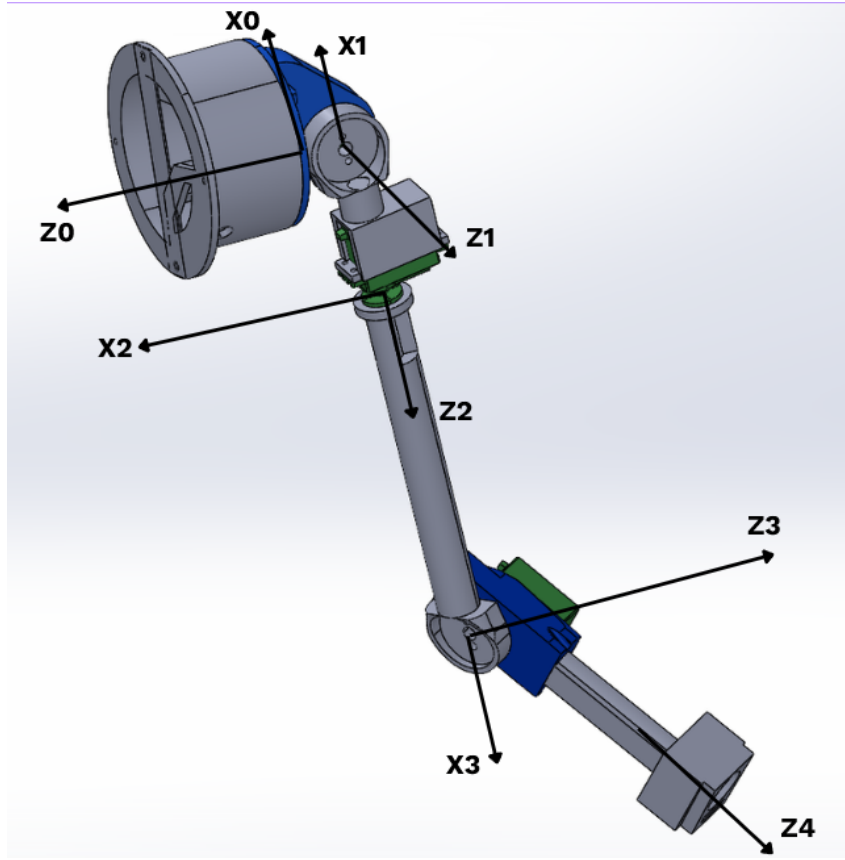


Figure 14: Coordinate frames and joint variables used in the kinematic model.

5.2. Forward Kinematics

The forward kinematics of the robotic arm was developed using a direct rotation–translation transformation method rather than the conventional Denavit–Hartenberg (DH) convention. This approach was selected because the three shoulder joint axes are physically separated and do not intersect at a single point. Therefore, the actual three-dimensional offsets between the joint centres were measured and incorporated directly into the kinematic model.

The joint variable vector is defined as

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix}^T, \quad (2)$$

where q_1 , q_2 , q_3 , and q_4 represent the shoulder pitch, shoulder roll, shoulder yaw, and elbow pitch angles, respectively. All joint angles are defined relative to the hanging rest pose of the arm.

The body-fixed world coordinate system is defined with its origin at the centre of the supporting base. The coordinate directions are defined as

$$+x : \text{forward}, \quad +y : \text{left/outward}, \quad +z : \text{upward}. \quad (3)$$

The position of the shoulder pitch joint relative to the world origin is

$$\mathbf{b}_0 = \begin{bmatrix} 0 & 70 & 515 \end{bmatrix}^T \text{ mm}. \quad (4)$$

The measured offset vectors between consecutive joint centres are

$$\mathbf{p}_{01} = \begin{bmatrix} 0 & 36 & -14 \end{bmatrix}^T \text{ mm}, \quad (5)$$

$$\mathbf{p}_{12} = \begin{bmatrix} 0 & 0 & -85 \end{bmatrix}^T \text{ mm}, \quad (6)$$

$$\mathbf{p}_{23} = \begin{bmatrix} 0 & 29 & -205 \end{bmatrix}^T \text{ mm}, \quad (7)$$

$$\mathbf{p}_{34} = \begin{bmatrix} 0 & 25 & -181 \end{bmatrix}^T \text{ mm}. \quad (8)$$

Here, $\mathbf{p}_{01}$ represents the offset from the shoulder pitch joint to the shoulder roll joint, $\mathbf{p}_{12}$ represents the offset from the shoulder roll joint to the shoulder yaw joint, $\mathbf{p}_{23}$ represents the offset from the shoulder yaw joint to the elbow joint, and $\mathbf{p}_{34}$ represents the offset from the elbow joint to the end-effector reference point.

The geometric parameters used in the forward kinematic model are summarised in Table 3.

Table 3: Measured geometric offsets used in the forward kinematic model.

Transformation	Associated rotation	Offset vector (mm)
World to shoulder pitch	–	$\mathbf{b}_0 = [0, 70, 515]^T$
Shoulder pitch to shoulder roll	$\mathbf{R}_y(-q_1)$	$\mathbf{p}_{01} = [0, 36, -14]^T$
Shoulder roll to shoulder yaw	$\mathbf{R}_x(q_2)$	$\mathbf{p}_{12} = [0, 0, -85]^T$
Shoulder yaw to elbow	$\mathbf{R}_z(q_3)$	$\mathbf{p}_{23} = [0, 29, -205]^T$
Elbow to end-effector	$\mathbf{R}_y(-q_4)$	$\mathbf{p}_{34} = [0, 25, -181]^T$

The elementary rotation matrices are defined as

$$\mathbf{R}_x(q) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos q & -\sin q \\ 0 & \sin q & \cos q \end{bmatrix}, \quad (9)$$

$$\mathbf{R}_y(q) = \begin{bmatrix} \cos q & 0 & \sin q \\ 0 & 1 & 0 \\ -\sin q & 0 & \cos q \end{bmatrix}, \quad (10)$$

and

$$\mathbf{R}_z(q) = \begin{bmatrix} \cos q & -\sin q & 0 \\ \sin q & \cos q & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (11)$$

The negative signs applied to q_1 and q_4 account for the physical installation direction of the shoulder pitch and elbow servos on the left robotic arm. Under the adopted convention, a positive shoulder pitch angle raises the arm forward, while a positive elbow angle bends the elbow.

The cumulative orientation matrices are calculated sequentially as

$$\mathbf{R}_1 = \mathbf{R}_y(-q_1), \quad (12)$$

$$\mathbf{R}_2 = \mathbf{R}_1 \mathbf{R}_x(q_2), \quad (13)$$

$$\mathbf{R}_3 = \mathbf{R}_2 \mathbf{R}_z(q_3), \quad (14)$$

$$\mathbf{R}_4 = \mathbf{R}_3 \mathbf{R}_y(-q_4). \quad (15)$$

The end-effector position in the world coordinate system is subsequently obtained from

$$\mathbf{p}_{EE} = \mathbf{b}_0 + \mathbf{R}_1\mathbf{p}_{01} + \mathbf{R}_2\mathbf{p}_{12} + \mathbf{R}_3\mathbf{p}_{23} + \mathbf{R}_4\mathbf{p}_{34}. \quad (16)$$

Equivalently, the complete kinematic chain may be expressed using homogeneous transformations as

$${}^W\mathbf{T}_{EE} = \mathbf{T}(\mathbf{b}_0)\mathbf{R}_y^H(-q_1)\mathbf{T}(\mathbf{p}_{01})\mathbf{R}_x^H(q_2)\mathbf{T}(\mathbf{p}_{12})\mathbf{R}_z^H(q_3)\mathbf{T}(\mathbf{p}_{23})\mathbf{R}_y^H(-q_4)\mathbf{T}(\mathbf{p}_{34}), \quad (17)$$

where $\mathbf{T}(\mathbf{p})$ is a homogeneous translation matrix and $\mathbf{R}_x^H$, $\mathbf{R}_y^H$, and $\mathbf{R}_z^H$ are the corresponding 4×4 homogeneous rotation matrices.

At the rest configuration,

$$\mathbf{q} = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T, \quad (18)$$

the predicted end-effector position is

$$\mathbf{p}_{EE,0} = \mathbf{b}_0 + \mathbf{p}_{01} + \mathbf{p}_{12} + \mathbf{p}_{23} + \mathbf{p}_{34} \quad (19)$$

$$= \begin{bmatrix} 0 & 160 & 30 \end{bmatrix}^T \text{ mm}. \quad (20)$$

In the ESP32 firmware, the forward kinematics is implemented in the `calculateFK()` function. The function sequentially updates the cumulative rotation matrix and transforms each measured offset vector into the world frame. The transformed vectors are then added to obtain the Cartesian end-effector position,

$$\mathbf{p}_{EE} = \begin{bmatrix} x_{EE} & y_{EE} & z_{EE} \end{bmatrix}^T. \quad (21)$$

The implementation uses the custom `Vec3` and `Mat3` data structures for three-dimensional vector and rotation-matrix calculations. This method directly represents the measured geometry of the physical prototype and avoids introducing artificial DH frames for the spatially separated shoulder joint axes.

5.3. Inverse Kinematics

The inverse kinematics of the robotic arm is solved numerically rather than using a closed-form geometric solution. This approach is consistent with the forward-kinematic model described in the previous section, which directly uses the measured three-dimensional offsets between the joint axes. Because the shoulder pitch, roll, and yaw axes are spatially separated, reducing the arm to a simple planar two-link mechanism would not accurately represent the physical prototype.

The desired end-effector position is defined in the body-fixed world coordinate system as

$$\mathbf{p}_d = \begin{bmatrix} x_d & y_d & z_d \end{bmatrix}^T. \quad (22)$$

The joint-angle vector is

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix}^T, \quad (23)$$

where q_1 , q_2 , q_3 , and q_4 correspond to shoulder pitch, shoulder roll, shoulder yaw, and elbow pitch, respectively.

Let the forward-kinematic function be denoted by

$$\mathbf{p}_{EE} = \mathbf{f}(\mathbf{q}), \quad (24)$$

where $\mathbf{f}(\mathbf{q})$ is evaluated using the measured-offset rotation-matrix model implemented in the `calculateFK()` function.

The inverse-kinematics problem is formulated as the minimisation of the Cartesian position error

$$\mathbf{e}(\mathbf{q}) = \mathbf{f}(\mathbf{q}) - \mathbf{p}_d. \quad (25)$$

The corresponding objective function is

$$J(\mathbf{q}) = \|\mathbf{f}(\mathbf{q}) - \mathbf{p}_d\|^2, \quad (26)$$

or, equivalently,

$$J(\mathbf{q}) = (x_{EE} - x_d)^2 + (y_{EE} - y_d)^2 + (z_{EE} - z_d)^2. \quad (27)$$

The numerical inverse-kinematics solver searches for

$$\mathbf{q}^* = \arg \min_{\mathbf{q}} J(\mathbf{q}), \quad (28)$$

subject to the physical joint limits

$$-50^\circ \leq q_1 \leq 130^\circ, \quad (29)$$

$$0^\circ \leq q_2 \leq 160^\circ, \quad (30)$$

$$-30^\circ \leq q_3 \leq 150^\circ, \quad (31)$$

$$-55^\circ \leq q_4 \leq 125^\circ. \quad (32)$$

A multistart coordinate-descent search is used to reduce the likelihood of the solver converging to an unsuitable local minimum. Several initial joint configurations are tested, including the current arm configuration, the rest configuration, and a set of representative bent-arm poses. The solution that produces the smallest final Cartesian error is selected.

For each initial configuration, the solver adjusts one joint at a time. For joint q_i , two candidate values are evaluated:

$$q_i^+ = q_i + \Delta q, \quad (33)$$

$$q_i^- = q_i - \Delta q. \quad (34)$$

The forward kinematics is evaluated for both candidates, and the candidate that produces the smallest value of $J(\mathbf{q})$ is retained. If neither candidate reduces the objective function, the original joint angle is preserved.

The search is first performed using a large angular step and is then refined using progressively smaller step sizes:

$$\Delta q \in \{30^\circ, 15^\circ, 8^\circ, 4^\circ, 2^\circ, 1^\circ, 0.5^\circ\}. \quad (35)$$

This coarse-to-fine procedure allows the solver to first move rapidly toward the target region and then improve the final end-effector accuracy. At every iteration, the candidate joint angles are constrained to their corresponding joint limits before the forward kinematics is evaluated.

The complete numerical procedure is summarised as follows:

1. Define the desired end-effector position $\mathbf{p}_d = [x_d, y_d, z_d]^T$.
2. Select an initial joint-angle configuration.

3. Calculate the current end-effector position using the forward kinematics:

$$\mathbf{p}_{EE} = \mathbf{f}(\mathbf{q}).$$

4. Calculate the squared Cartesian position error using Equation (26).
5. For each joint, test both $q_i + \Delta q$ and $q_i - \Delta q$.
6. Retain the candidate angle only when it reduces the Cartesian error.
7. Repeat the joint-by-joint search until no further improvement is obtained for the current step size.
8. Reduce the angular step size and repeat the search.
9. Repeat the process for multiple initial configurations.
10. Select the solution with the smallest final Cartesian error.

After the best solution has been identified, the final Euclidean position error is calculated as

$$e_f = \|\mathbf{f}(\mathbf{q}^*) - \mathbf{p}_d\|. \quad (36)$$

In the current firmware, the inverse-kinematics solution is accepted when

$$e_f \leq 35 \text{ mm}. \quad (37)$$

If the final error exceeds this threshold, the target is considered unreachable or insufficiently accurate, and the arm is not commanded to move.

The accepted mathematical joint angles are converted into raw servo commands using

$$s_i = s_{i,0} + d_i q_i, \quad (38)$$

where s_i is the commanded servo angle, $s_{i,0}$ is the servo angle at the rest configuration, and d_i represents the installed servo direction. For the current left-arm configuration, the direction coefficients are

$$\begin{bmatrix} d_1 & d_2 & d_3 & d_4 \end{bmatrix} = \begin{bmatrix} -1 & +1 & -1 & -1 \end{bmatrix}. \quad (39)$$

The raw servo commands are subsequently constrained to the mechanical limits of each actuator before the arm is moved.

In the ESP32 firmware, the numerical inverse kinematics is implemented through the `solveIK()`, `solveIKFromSeed()`, and `positionErrorSq()` functions. The `solveIKFromSeed()` function performs the coordinate-descent search from one initial configuration, while `solveIK()` evaluates several initial configurations and selects the solution with the lowest final error.

The current inverse-kinematics implementation controls only the Cartesian position of the end-effector. End-effector orientation is not included in the objective function. This is sufficient for the initial reaching and tool-pickup experiments; however, orientation constraints will be required in future work for tasks such as drilling, tool docking, and precise placement of interchangeable end-effectors. ““

5.4. Joint-to-Servo Mapping

The joint angles computed by the kinematics engine must be converted to physical servo angles before being sent to the ESP32Servo library. The calibration equation for each actuator is:

The calibration equation for each actuator is

$$\theta_{\text{servo},i} = \theta_{\text{rest},i} + s_i q_i, \quad (40)$$

where:

- $\theta_{\text{rest},i}$ is the servo rest angle.
- q_i is the calculated joint displacement.
- s_i is the direction coefficient.
- $s_i = +1$ when the servo direction matches the model.
- $s_i = -1$ when the servo direction is reversed.

Calibration was performed by commanding each joint to its reference configuration and adjusting the rest angle until the measured arm pose matched the expected geometric configuration. Joint limits were enforced in software to prevent the arm from reaching configurations that could cause structural damage or self-collision.

5.5. Tool-Change Path Planning

All arm motion during a tool-change sequence is performed along predefined joint-space trajectories connecting a set of fixed waypoints. The use of predefined poses, rather than on-line Cartesian planning, ensures that the arm follows a known, collision-free path to the rotating tool station on every exchange cycle. The waypoints are:

1. Home pose: the resting configuration of the arm between operations.
2. Station approach pose: an intermediate pose that positions the arm in front of the station without risk of collision.
3. Alignment pose: the configuration in which the wrist alignment guide engages the tool holder, correcting residual lateral error.
4. Attachment or release pose: the final approach configuration in which the wrist face contacts the tool steel plate and the electromagnet is activated or deactivated.
5. Retreat pose: an intermediate configuration used after tool pickup to clear the station before returning to the home pose.

The joint angles between two poses may be interpolated using

$$q_i(k) = q_{i,0} + \frac{k}{N} (q_{i,f} - q_{i,0}), \quad (41)$$

where:

- $q_{i,0}$ is the initial joint angle.
- $q_{i,f}$ is the final joint angle.
- N is the number of interpolation steps.

- $k = 0, 1, \dots, N$.

This produces a smooth, coordinated motion in which all joints begin and end their motion simultaneously. The interpolation step count N and the inter-step delay are configurable in the firmware through the `moveAllSlow()` function, which allows the motion speed to be adjusted from the web interface without modifying the waypoint definitions.

6. Electrical and Embedded System

6.1. Electronic Components

The main electrical components are summarized in Table 4.

Table 4: Main electrical components of the final prototype.

Component	Quantity	Function
ESP32 development board	1	Main controller and web server
High-torque servo motors	4	Shoulder and elbow actuation
NEMA 17 stepper motor	1	Rotation of the tool station
A4988 stepper driver	1	Stepper motor current and direction control
Relay modules	2	Electromagnet and tool power switching
Electromagnet	1	End-effector attachment
Pogo-pin connector	3 set	Electrical power transfer to the attached tool
Tool actuators	1	Actuation of the interchangeable tools

6.2. Power Distribution and Hardware Connections

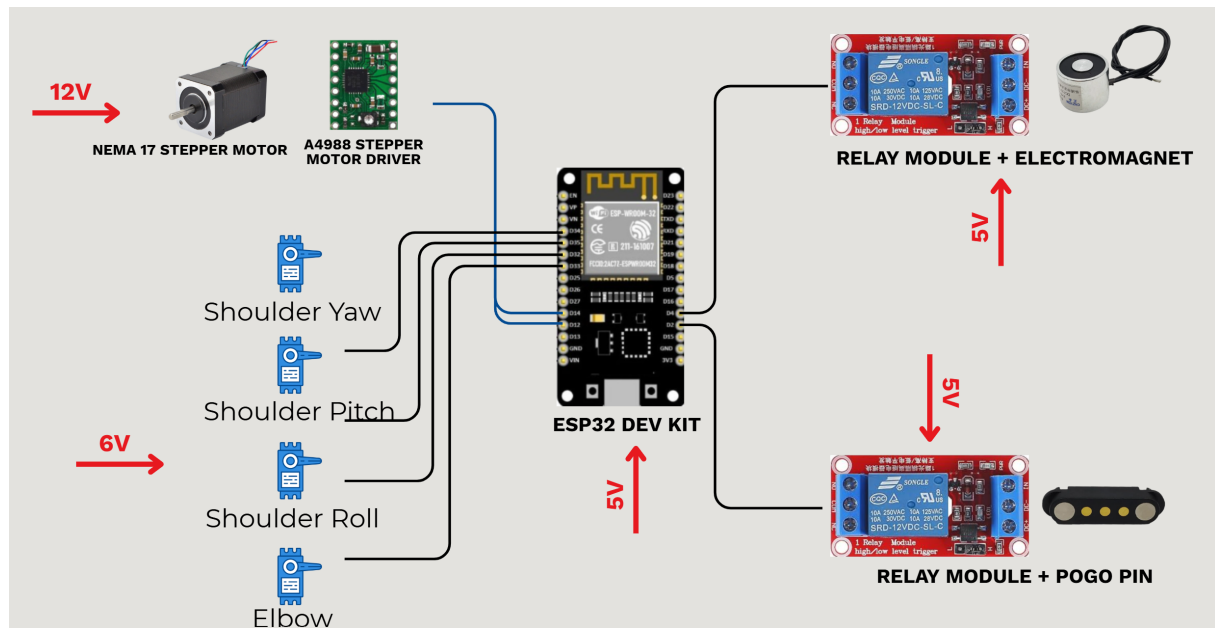


Figure 15: Power and Hardware wiring diagram.

The power distribution and wiring diagram are shown in Fig.15. The system uses 3 different voltage value:

- 12V for the stepper motor and electromagnet/
- 6V for the robot arm motors.
- 5V for the relay modules and ESP32.

For the simplicity of the project, we will utilize the Keysight DC voltage supply to provide different voltages. The whole system could be totally adjusted to use step up/step down voltage IC if planning to optimize for industrial usage. All components are checked to ensure common ground.

6.3. Electromagnet and Pogo-Pin Interface

The electromagnet is controlled through a relay for the ON/OFF mechanism. When the control signal is active, the electromagnet is energized and the tool is pulled toward the wrist mounting surface.

The pogo pins establish an electrical connection after the mechanical attachment is completed. The controller should activate tool power only after the tool is securely attached.

6.4. Safety and Tuning Considerations

The electrical and embedded design should include the following requirements:

- All tools remain OFF during startup.
- Tool power is disabled during the exchange process.
- Joint angles are limited in software.
- The stepper motor movement is limited to valid station positions.
- The electromagnet should remain active during tool operation.
- An emergency stop or software stop should be available.
- The drill or saw tool should include a protective cover.
- High-current devices should use suitable wiring and power supplies.

7. Control Software and Web Interface

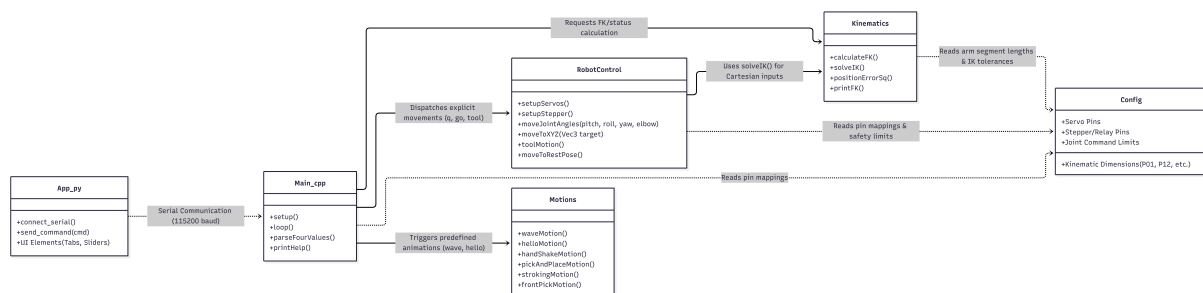


Figure 16: Overall block diagram of the code.

7.1. Firmware Structure

7.1.1 Configuration & Constants (config.h)

This file acts as the single source of truth for all physical and hardware-specific parameters. Centralizing these values makes it easy to calibrate the robot without digging through logic code. It defines:

- **Hardware Mapping:** GPIO pins for the 4 joint servos, the tool servo, the stepper motor, and the electromagnet relay.
- **Safety Limits & Offsets:** Minimum/maximum raw PWM limits, servo safety angle constraints (e.g., preventing the arm from colliding with itself), and calibration offsets for rest poses.
- **Kinematic Measurements:** The physical lengths and offsets between joints (P01, P12, P23, P34) in a 3D coordinate space, which are critical for accurate mathematical calculations.

7.1.2 Hardware Abstraction & Control (robot_control.cpp / .h)

This module is responsible for safely moving the physical actuators. It translates mathematical angles into electronic signals.

- **Servo Management:** Uses the `ESP32Servo` library to allocate hardware PWM timers and send precise pulses to the 5 servos.
- **Smooth Motion Profiling:** Instead of snapping servos instantly to target angles (which causes violent jerks), functions like `moveAllSlow()` and `moveJointAngles()` increment the angles step-by-step with a configurable delay (`stepDelayMs`).
- **Safety Clamping:** Before any movement is executed, the requested angles are passed through `clampJoint()` and `clampServoAngle()` to ensure they do not exceed the mechanical limits defined in `config.h`.

7.1.3 Mathematics & Kinematics Engine (kinematics.cpp / .h)

This is the “brain” of the arm, responsible for translating between 3D space (Cartesian coordinates) and joint angles. It implements custom matrix mathematics (`Vec3`, `Mat3`, Rotations).

- **Forward Kinematics (FK):** The `calculateFK()` function takes the 4 joint angles (pitch, roll, yaw, elbow) and applies 3D rotation matrices to calculate exactly where the end-effector (tool) is in physical space (X, Y, Z).
- **Inverse Kinematics (IK):** The `solveIK()` function allows the user to input a desired X, Y, Z target. Because finding the exact angles for a 4DOF arm is complex, the firmware uses an **iterative heuristic solver** (`solveIKFromSeed`). It starts from several “seed” poses (including the current pose and preset guesses) and incrementally tweaks the angles using step sizes to minimize the spatial error (`positionErrorSq`) until it reaches the target within an acceptable tolerance (`IK.ACCEPT_ERROR_MM`).

7.1.4 Behavioral Logic (motions.cpp / .h)

This module stores complex, pre-programmed animation sequences.

- Instead of requiring the user to send dozens of commands to perform a task, functions like `waveMotion()`, `pickAndPlaceMotion()`, and `handShakeMotion()` chain together multiple `moveJointAngles()` and `toolMotion()` calls with specific timing delays.

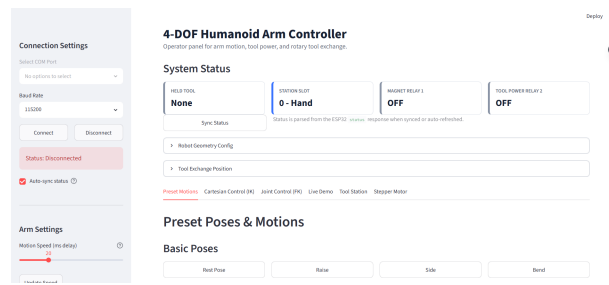


Figure 17: Web UI main page

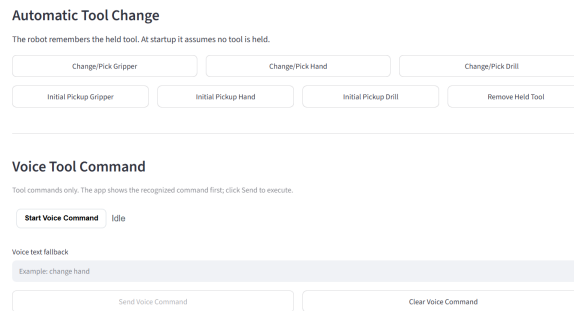


Figure 18: Web UI Tool Control

7.1.5 Main Loop & Serial CLI (main.cpp)

The main entry point acts as a Command Line Interface (CLI) over the ESP32's Serial connection.

- **Initialization:** The `setup()` function initializes the servos, stepper pins, relay, and prints a comprehensive help menu to the user.
- **Command Parsing:** The `loop()` listens for incoming string commands via Serial at 115200 baud. It parses these strings (e.g., extracting numbers from a string like `go 120 160 250` using `parseThreeValues()`) and routes the instruction to the correct subsystem.
- **Dispatching:**
 - If the user types `wave`, it calls `waveMotion()`.
 - If the user types `q 90 90 0 45`, it routes directly to `moveJointAngles()` in the control module.
 - If the user types `go 120 150 200`, it sends the coordinates to `moveToXYZ()`, which triggers the IK solver in the kinematics module, and if successful, moves the arm to grab that point in space.

7.2. Web Interface

The interface is organized into a sidebar for general settings and a tabbed main area for different control methodologies. Here is a breakdown of its structure and features:

7.2.1 Sidebar (Global Settings & Connections)

The sidebar is persistent and handles the hardware connection and global robot states:

- **Connection Settings:** Allows the user to select the appropriate COM port and Baud Rate (defaulting to 115200 to match the ESP32 firmware) and click “Connect” or “Disconnect.”

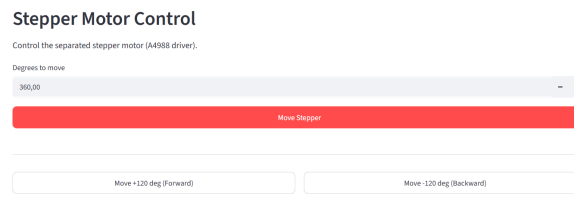


Figure 19: Web UI stepper control

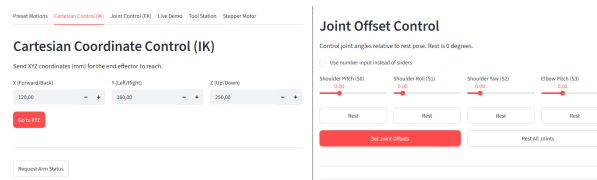


Figure 20: Web UI motion control

- **Arm Settings:** Features a slider to adjust the motion speed (delay in milliseconds). Smaller values result in faster motion.
- **Relay Control:** Provides dedicated buttons to manually turn the electromagnet relay ON or OFF.

7.2.2 Main Content Tabs

The main interface is divided into four functional tabs, allowing the user to choose how they want to command the robot:

- **Tab 1: Preset Motions**
 - Offers a grid of quick-action buttons to trigger the pre-programmed animations defined in the firmware.
 - Includes basic postural tests (Rest Pose, Raise, Side, Bend) and complex sequences (Wave, Hello, Handshake, Stroke, Pick & Place, Front Pick).
 - Also includes a button to quickly toggle the tool/gripper.
- **Tab 2: Cartesian Control (IK)**
 - Provides number input fields for X, Y, and Z Cartesian coordinates (in millimeters).
 - Clicking “Go to XYZ” sends a `go <x> <y> <z>` command to the ESP32, which utilizes the Inverse Kinematics (IK) engine to calculate the necessary joint angles to reach that point in space.
- **Tab 3: Joint Control (FK)**
 - Provides direct control over the raw angles of each of the four servos (Shoulder Pitch, Roll, Yaw, and Elbow Pitch).
 - Uses both number inputs and sliders synced together for intuitive control.
 - Clicking “Set Servo Angles” sends an `all <s0> <s1> <s2> <s3>` command, moving the arm using Forward Kinematics.
- **Tab 4: Stepper Motor**
 - Dedicated to the auxiliary stepper motor control.

- Allows the user to input a specific number of degrees to move (positive for forward, negative for backward) or use quick-action buttons to move exactly +120 or -120 degrees.

At the very bottom of the interface, there is a “Request Arm Status” button. When clicked, it sends a `status` command to the ESP32, which prompts the microcontroller to output its current joint angles and calculated end-effector position (which can be read from the terminal/serial monitor).

Joint Control

The user may control each joint independently through sliders or numerical inputs. Before sending a command, the requested angle should be restricted to the allowed range:

$$q_{i,\text{cmd}} = \min[q_{i,\text{max}}, \max(q_{i,\text{min}}, q_{i,\text{input}})] . \quad (42)$$

7.3. Preset Motions

The final preset motions including:

- Rest pose.
- Home pose.
- Raised-arm pose.
- Side pose.
- Bend pose.
- Wave motion.
- Hello motion.
- Handshake motion.
- Forward stroke.
- Pick-and-place motion.
- Front-pick motion.
- Tool approach pose.
- Tool-change sequence.

7.4. Automatic Tool-Change State Machine

The automatic tool-change state sequence is:

1. IDLE
2. MOVE_TO_APPROACH
3. ALIGN_WITH_STATION
4. RELEASE_CURRENT_TOOL
5. RETRACT_FROM_STATION
6. ROTATE_STATION

7. APPROACH_TARGET_TOOL
8. ATTACH_TARGET_TOOL
9. VERIFY
10. RETRACT
11. COMPLETE
12. FAULT

8. Experimental Methodology

8.1. Test Setup

The testing setup including

- Robotic arm.
- Tool station.
- ESP32 controller.
- Power supplies.
- Computer or mobile device.
- Web interface.
- Camera.
- Measuring tools (Tape Measure, Caliper, Level Ruler)
- Test objects.

8.2. Arm Motion Test

The arm motion test evaluates whether the arm can move safely and repeatedly between the rest pose, preset gesture poses, and the tool exchange pose. The configured rest pose is

$$q_{\text{rest}} = [0, 0, 0, 50]^T, \quad (43)$$

and the configured tool exchange pose is

$$q_{\text{exchange}} = [-3, 0, 0, -3]^T. \quad (44)$$

The following tests has been performed:

- Move each joint individually using the joint control interface.
- Verify that each joint remains within its configured software limit.
- Command the arm to the rest pose and confirm stable positioning.
- Command the arm to the tool exchange pose and confirm that it can reach the tool station.
- Run the preset motions, including hello, wave, handshake, and showcase.
- Observe the smoothness of motion during slow docking and tool removal.
- Check for vibration, excessive flexing, or instability during tool exchange.
- Repeat the same preset motion several times and observe whether the final pose is consistent.

8.3. Tool Station Indexing Test

The rotating tool station contains three indexed tool locations:

- Slot 0: static hand model.
- Slot 1: drill.
- Slot 2: gripper.

The station is commanded to each slot repeatedly using both manual station commands and automatic tool exchange commands. The standard angular spacing between tools is

$$\theta_{\text{slot}} = 120^\circ. \quad (45)$$

For the showcase sequence, the final tool indexing motion is adjusted to 115° to compensate for observed accumulated alignment error.

8.4. Attachment and Release Test

The attachment and release test evaluates the magnetic tool locking process and the mechanical docking reliability of the tool station. During pickup, the arm moves to a prechange pose, approaches the tool, oscillates the station for alignment, enables the magnet, performs a small settling shake, and then moves out with the tool.

During return, the tool is powered off first. If the gripper is held, the gripper is closed before returning. The arm then approaches the station, the station performs an oscillation before final docking, the magnet is disabled, and the arm moves away without the tool.

The following are satisfy:

- Successful pickup for the hand, drill, and gripper.
- Successful return for the hand, drill, and gripper.
- Magnet attaches before the tool is lifted.
- Tool remains attached during the move-out motion.
- Tool releases cleanly after magnet deactivation.
- Effectiveness of the station oscillation during pickup and return.
- Repeatability during the full showcase sequence.

8.5. Electrical Contact Test

The electrical contact test verifies the reliability of the powered tool interface. The hand model is passive and does not require tool power. The gripper and drill use the tool power relay through the pogo-pin connection.

- Tool power relay operation from the web interface.
- Correct power-off behavior before returning a tool.
- Stable gripper servo operation after pickup.
- Stable drill operation during the configured showcase run time.
- Absence of intermittent power loss during arm movement.

- Contact reliability after repeated pickup and return cycles.
- Correct status reporting of magnet state, tool power state, active tool, and station slot.

For safety, the static hand tool should not enable tool power. The drill should only remain powered during commanded tool operation, and the gripper should only be powered during open or close commands.

9. Challenges, Limitations, and Improvements

9.1. Mechanical Challenges

Several mechanical challenges were observed during development of the arm and automatic tool station. The most important issue was repeatable alignment between the end effector and the rotating tool station. Even when the arm could reach the nominal exchange pose, small errors in the station angle, printed-part tolerance, or tool seating position caused the tool to miss the magnetic pickup point.

The main mechanical challenges were:

- Small tool-center misalignment between the arm magnet and the docked tool.
- Accumulated indexing error after multiple tool changes.
- Tool-station angle error during the final tool in the showcase sequence.
- Printed-part tolerance errors in the docking geometry.
- Limited stiffness of the tool station during docking and release.
- Arm deflection when carrying heavier tools.
- The vacuum tool was too heavy and risky for reliable operation.
- Gripper opening and closing required tuning to avoid unstable motion.
- The docking motion applied too much force when oscillation occurred at the final exchange pose.

To reduce docking error, the tool exchange procedure was modified several times. The final procedure uses a prechange pose, a slower final elbow approach, station oscillation, magnet attachment, and a small settling shake. For return motion, the station oscillation was moved away from the final dock pose to reduce force on the tool station. For pickup motion, a separate partial oscillation pose was added so the station can wiggle before the magnet locks onto the tool.

9.2. Electrical Challenges

The electrical system also required tuning because the tool interface uses relays, a magnet, servo power, and pogo-pin style tool power. The gripper servo was not stable when it was powered and commanded immediately, so a power-settling delay was added before sending servo commands.

The main electrical challenges were:

- Tool power relay timing caused unstable gripper behavior.
- The gripper servo needed time to finish each open or close movement.
- The electromagnet needed a delay after activation before lifting the tool.

- The web interface originally did not show the real magnet and tool-power states.

To improve reliability, firmware state variables were added for magnet and tool power status. These values are now reported through the status command and parsed by the web interface. Tool power is disabled before return, and the gripper is closed before it is returned to the station.

9.3. Control Challenges

The control system was developed through several iterations. The arm originally treated the raw servo calibration pose as the rest pose, which made startup appear close to the tool exchange pose. This was corrected by separating the normal rest pose from the tool exchange pose.

The main control challenges were:

- The startup pose initially used raw servo rest angles instead of the configured arm rest pose.
- The rest pose and tool exchange pose needed to be separated.
- The docking approach required several changes before a stable sequence was found.
- The station slot order changed after replacing the vacuum tool with a passive hand model.
- The web interface did not stay synchronized with firmware state.
- Voice command transfer from the browser microphone to Streamlit was limited by browser iframe behavior.
- The showcase sequence accumulated enough error that the final station rotation needed compensation.

The final control procedure uses tool-specific commands for pickup, return, showcase, relay control, and status synchronization. The web interface now parses firmware status output to update the active tool, station slot, magnet state, tool power state, and current joint pose.

9.4. Solutions Implemented

The major problems and implemented solutions are summarized in Table 5.

Table 5: Challenges and implemented solutions.

Observed problem	Root cause	Implemented solution
Tool sometimes missed during pickup	Small docking and station-indexing errors	Added station oscillation before pickup, magnet-on before settling shake, and a slower final docking motion
Tool return placed too much force on the station	Oscillation occurred at the final exchange pose	Moved return oscillation to a partial return pose before final docking
The final tool in the showcase sequence was slightly misaligned	Accumulated station indexing error after repeated exchanges	Added a showcase-only final station rotation of 115° instead of the normal 120°
Vacuum tool was unsafe and unreliable	Tool was too heavy for the prototype arm and increased docking risk	Replaced the vacuum tool with a passive static hand model
Gripper servo behaved unstably	Servo was commanded immediately after relay power was enabled	Added a tool-power settling delay and gripper movement delay
Robot appeared to start at the changing pose	Firmware initialized servos to raw calibration pose instead of configured rest pose	Added a separate arm rest pose $[0, 0, 0, 50]^T$ and initialized startup to this pose
Tool exchange pose conflicted with rest pose	Rest and exchange were treated too similarly during early testing	Defined the tool exchange pose separately as $[-3, 0, 0, -3]^T$
Web dashboard did not match real robot state	The app only remembered sent commands and did not parse firmware status	Added firmware status output for active tool, station slot, magnet state, tool power, and current joint pose
Voice command could not directly execute reliably	Browser speech recognition runs inside an embedded Streamlit component	Added a tool-only voice command panel with recognized-text mapping and manual confirmation before sending

9.5. Current Limitations

The current prototype still has several limitations:

- Tool exchange is still mostly open-loop.
- There is no vision-based alignment correction.
- There is no force or torque feedback at the wrist.
- The rotating station has no absolute encoder.
- Station alignment depends on calibrated stepper motion.
- The showcase sequence still requires a custom final-angle compensation.
- The electromagnet does not provide feedback confirming attachment.
- The arm has limited payload capacity, which prevented safe use of the vacuum tool.

- The voice command interface depends on browser support for speech recognition.
- Voice commands require manual confirmation before sending.
- The system depends on predefined rest, prechange, and exchange poses.
- The rotating drill tool still requires careful safety handling.

10. Project Management and Team Contributions

10.1. Development Timeline

Table 6: Final project development timeline.

Period	Completed work
Weeks 1–2	Requirement definition, concept development, and system architecture
Weeks 3–4	Robotic arm and tool-coupling CAD design
Weeks 5–6	Kinematic modeling, MATLAB simulation, and electronic testing
Weeks 6–7	3D printing and mechanical assembly
Weeks 7–8	ESP32 firmware, servo control, and tool-station control
Weeks 8–9	Web-interface development and automatic sequence implementation
Weeks 9–10	Integration, calibration, experimental testing, and final demonstration

10.2. Team Contributions

Table 7: Final team contributions.

Team member	Final contribution
Le Hai Trung	<i>3D design, manufacturing, system integration, robotic-arm motion, kinematic implementation, and final validation</i>
Ngo Hai Khanh	<i>Electrical system, firmware, web interface, control algorithm, and system integration</i>
Doan Chi Dung	<i>Mechanical design, wrist coupling, tool station, fabrication, assembly, and testing</i>

11. Conclusion and Future Work

This project developed a four-degree-of-freedom humanoid robotic arm with a modular end-effector interface for interchangeable tools. The arm was modelled using measured joint offsets and sequential rotation matrices, while a numerical inverse-kinematics solver was implemented to determine suitable joint angles for Cartesian target positions. Experimental motions demonstrated the ability of the arm to reach, perform predefined tasks, and activate an attached tool through serial commands. Future work will focus on improving positioning accuracy, adding end-effector orientation control, and developing an automatic magnetic locking and tool-docking mechanism for reliable switching between suction, gripper, and drilling tools.

11.1. Future Work

Future work may include:

- Automatic tool recognition.
- Closed-loop vision-based docking.
- Force and torque sensing.
- Improved mechanical locking.
- Encoder-based station control.
- Higher-payload actuators.
- Collision detection.
- Integration with a complete humanoid platform.
- Autonomous task-level planning.
- Tool selection based on object and task recognition.

Acknowledgment

The authors would like to thank the course instructor, laboratory staff, and all individuals who supported the design, fabrication, component procurement, testing, and validation of this project.

References

- [1] Unitree Robotics, *Unitree G1 Documentation*, <https://github.com/unitreerobotics>.
- [2] T. M. Thanh, *ELEC4060 / COMP5050 Robotics Course Notes*, Vin University, 2026.
- [3] X. Jiang, B. Wu, S. Li, Y. Zhu, G. Liang, Y. Yuan, Q. Li, and J. Zhang, “Multi-Humanoid Robot Arm Motion Imitation and Collaboration Based on Improved Retargeting.”